Several Instances of a Resource Type

- The wait-for graph scheme is not applicable to a resource-allocation system with multiple instances of each resource type. We turn now to a deadlock-detection algorithm that is applicable to such a system. The algorithm employs several time-varying data structures that are similar to those used in the banker's algorithm (Section 7.5.3):

Several Instances of a Resource Type

- Available. A vector of length m indicates the number of available resources of each type.
- Allocation. An n × m matrix defines the number of resources of each type currently allocated to each process.
- Request. An n × m matrix indicates the current request of each process. If Request[i][j] equals k, then process Pi is requesting k more instances of resource type Rj .

Several Instances of a Resource Type

1. Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available. For i = 0, 1, ..., n–1, if Allocationi ≠ 0, then Finish[i] = false. Otherwise, Finish[i] = true.
2. Find an index i such that both
   a. Finish[i] == false
   b. Requesti ≤ Work
   If no such i exists, go to step 4.
3. Work = Work + Allocationi
   Finish[i] = true
   Go to step 2.
4. If Finish[i] == false for some i, 0 ≤ i < n, then the system is in a deadlocked state. Moreover, if Finish[i] == false, then process Pi is deadlocked.

Several Instances of a Resource Type

- You may wonder why we reclaim the resources of process Pi (in step 3) as soon as we determine that Requesti ≤ Work (in step 2b). We know that Pi is currently not involved in a deadlock (since Requesti ≤ Work). Thus, we take an optimistic attitude and assume that Pi will require no more resources to complete its task; it will thus soon return all currently allocated resources to the system. If our assumption is incorrect, a deadlock may occur later. That deadlock will be detected the next time the deadlock-detection algorithm is invoked.

Several Instances of a Resource Type

- To illustrate this algorithm, we consider a system with five processes P0 through P4 and three resource types A, B, and C. Resource type A has seven instances, resource type B has two instances, and resource type C has six instances. Suppose that, at time T0, we have the following resource-allocation state:

	Allocation	***Request***	***Available***
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Several Instances of a Resource Type

- We claim that the system is not in a deadlocked state. Indeed, if we execute our algorithm,
- We will find that the sequence <P0, P2, P3, P1, P4>/ <P0, P2, P3, P4, P1> results in Finish[i] == true for all i.

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Several Instances of a Resource Type

- Suppose now that process P2 makes one additional request for an instance of type C. The Request matrix is modified as follows:

	Allocation	Request	Available
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	



	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

Several Instances of a Resource Type

- We claim that the system is now deadlocked. Although we can reclaim the resources held by process P0, the number of available resources is not sufficient to fulfill the requests of the other processes. Thus, a deadlock exists, consisting of processes P1, P2, P3, and P4.

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	



	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2